Домашнее задание 13 — BlockingQueue и Producer-Consumer

- **Дедлайн:** 30.10.2026
- **Тема:** BlockingQueue и Producer-Consumer
- **Форма проверки:** задание с самопроверкой
- **Вариант итогового ДЗ:** 3 — парсер вакансий
- **Модуль:** `hw13.queue` в Maven-проекте `app/`

---

1. Что требовалось и что сделано

Текст задания: реализовать функционал с `BlockingQueue` для передачи задач между производителями и потребителями — сервер задач, несколько producer'ов кладут URL в очередь, consumer'ы берут и обрабатывают (скачивают страницу и извлекают заголовок). Отчёт с этапами.

Сделано:

Что	Где
Задача и сигнал остановки	`model.UrlTask` (`UrlTask.POISON`)
Результат обработки страницы	`model.PageResult`
Ограниченная очередь с наблюдениями	`queue.TaskQueue`
Производитель	`queue.UrlProducer`
Потребитель	`queue.PageConsumer`
Загрузчик страниц	`web.PageFetcher`, `web.PageSource`
Извлечение `<title>`	`web.TitleExtractor`
Локальный сайт вакансий	`server.VacancySiteServer`
Сервер задач (оркестрация потоков)	`server.CrawlServer`
Итог прогона	`report.CrawlReport`
Строки отчёта	`report.QueueReport`
Демонстрация	`BlockingQueueDemo`, запуск `--app.hw=hw13`
Настройки `app.hw13.*`	`config.Hw13Properties`

Предметная область — парсер вакансий: производители кладут адреса страниц вакансий, потребители скачивают HTML и извлекают заголовок (`<title>`). Источник — локальный сайт на свободном порту, поэтому сеть не нужна; часть страниц отвечает `404`, чтобы проверить, что сбой страницы — это исход обработки, а не потерянная задача.

2. Этапы выполнения

Этап 1. Сервер задач и очередь

`CrawlServer.run(...)` собирает конвейер:

1. адреса разбиваются между производителями по кругу (`producer-i` берёт страницы `i, i+P, i+2P, …`);
2. **сначала стартуют потребители**, потом производители — иначе быстрый производитель заполнил бы небольшую очередь и заблокировался на пустом конвейере;
3. производители кладут `UrlTask` в `TaskQueue` (внутри — `LinkedBlockingQueue` **с ёмкостью**);
4. последний завершившийся производитель кладёт по одной «таблетке» `POISON` на каждого потребителя;
5. потребители берут задачи, скачивают страницу, извлекают заголовок и складывают `PageResult` в потокобезопасную очередь;
6. `CrawlServer` ждёт потоки с ограничением по времени; если кто-то не завершился — прерывает и помечает прогон как незавершённый.

Почему «таблетки», а не флаг «работа окончена»: флаг пришлось бы проверять в цикле, и потребитель мог бы завершиться, пока в очереди ещё лежат задачи. Сигнал стоит в очереди **после** всех задач, поэтому каждый потребитель сначала обрабатывает всё, что успело накопиться, и только потом останавливается.

Этап 2. Обратное давление: маленькая очередь

Ёмкость 4, 3 производителя, 3 потребителя, 24 страницы, задержка страницы 60 мс:

```
Этап 2. Маленькая очередь (4): производители ждут потребителей
    очередь на 4: подано 24, положено 24, взято 24, успешно 21, сбоев 3, глубина очереди 4 из
    без потерь: да, повторов страниц: нет, разных страниц 24 из 24, все потоки завершились: да
    распределение по потребителям: hw13-consumer-1=8, hw13-consumer-2=9, hw13-consumer-3=7
    сбои по кодам: код 404: 3
    примеры заголовков:
      №0 «Вакансия №0 — Java-разработчик» (поток hw13-consumer-1)
      №1 «Вакансия №1 — Backend-разработчик (Java)» (поток hw13-consumer-2)
      №2 «Вакансия №2 — Разработчик Spring Boot» (поток hw13-consumer-3)
      №3 «Вакансия №3 — Инженер по тестированию» (поток hw13-consumer-1)
      №5 «Вакансия №5 — Team Lead Java» (поток hw13-consumer-2)
    журнал «кто что делал» (первые 10 строк):
      [hw13-producer-3] положил задача №2 http://127.0.0.1:46647/vacancy/2
      [hw13-producer-3] положил задача №5 http://127.0.0.1:46647/vacancy/5
      [hw13-producer-3] положил задача №8 http://127.0.0.1:46647/vacancy/8
      [hw13-producer-3] положил задача №11 http://127.0.0.1:46647/vacancy/11
      [hw13-producer-3] положил задача №14 http://127.0.0.1:46647/vacancy/14
      [hw13-producer-1] положил задача №0 http://127.0.0.1:46647/vacancy/0
      [hw13-producer-2] положил задача №1 http://127.0.0.1:46647/vacancy/1
      [hw13-consumer-2] обработал задача №1 http://127.0.0.1:46647/vacancy/1 → «Вакансия №1 —
      [hw13-consumer-3] обработал задача №2 http://127.0.0.1:46647/vacancy/2 → «Вакансия №2 —
      [hw13-consumer-1] обработал задача №0 http://127.0.0.1:46647/vacancy/0 → «Вакансия №0 —
    ожидание производителей в put: 12 раз, 1669.0 мс
```

Наблюдения:

- очередь **ни разу не превысила ёмкость** (максимум 4 из 4) — это и есть ограниченная очередь;
- производители **12 раз ждали** освобождения места, суммарно 1669 мс; быстрота производителей ограничена скоростью потребителей, а не памятью;
- все 24 задачи обработаны, у каждой ровно один результат: потерь и повторов нет;
- 3 страницы с `404` попали в результаты как сбои, а не потерялись;
- работу разобрали все три потребителя (8/9/7) — очередь действительно распределяла задачи, а не отдала всё одному потоку.

Этап 3. Большая очередь и сравнение

```
Этап 3. Большая очередь (32): задачи копятся, производители не ждут
    очередь на 32: подано 24, положено 24, взято 24, успешно 21, сбоев 3, глубина очереди 24 и
    без потерь: да, повторов страниц: нет, разных страниц 24 из 24, все потоки завершились: да
    распределение по потребителям: hw13-consumer-1=7, hw13-consumer-2=8, hw13-consumer-3=9

Этап 4. Сравнение ёмкостей
    малая очередь (ёмкость 4): глубина 4, блокировок put 12, ожидание 1669.0 мс, 25.3 стр./с
    большая очередь (ёмкость 32): глубина 24, блокировок put 0, ожидание 0.0 мс, 30.7 стр./с
    вывод: ограниченная очередь включает обратное давление (put ждёт), большая — копит задачи,
```

Разница по времени (949 против 781 мс) — не «маленькая очередь медленнее» в общем случае: потребители всё равно обрабатывают 24 страницы по 60 мс, поэтому нижняя граница ≈ 24 × 60 / 3 = 480 мс. Маленькая очередь добавляет переключения контекста на каждом `put` / `take`, большая — позволяет потребителям работать без остановок, но ценой того, что весь список адресов оказывается в памяти. Для потока данных из millions адресов это неприемлемо, поэтому рабочая конфигурация — ограниченная очередь.

3. Фактический вывод демонстрации

```
mvn -pl app spring-boot:run -Dspring-boot.run.arguments="--app.hw=hw13 --server.port=0"
```

Прогон завершился с кодом `0`, живых потоков ДЗ 13 после завершения нет.

```
=== ДЗ 13. BlockingQueue: производители и потребители в конвейере обхода ===


Процесс: pid 128953
Настройки ДЗ 13:
    - страниц: 24, производителей: 3, потребителей: 3
    - ёмкость очереди: 4, для сравнения: 32
    - задержка страницы: 60 мс, таймаут запроса: 2000 мс
    - предел ожидания завершения потоков: 10000 мс

Этап 1. Локальный сайт вакансий
    порт 46647, страниц 24, из них с кодом 404: 3
    пример адреса: http://127.0.0.1:46647/vacancy/0
```

```
Этап 2. Маленькая очередь (4): производители ждут потребителей
    очередь на 4: подано 24, положено 24, взято 24, успешно 21, сбоев 3, глубина очереди 4 из
    без потерь: да, повторов страниц: нет, разных страниц 24 из 24, все потоки завершились: да
    распределение по потребителям: hw13-consumer-1=8, hw13-consumer-2=9, hw13-consumer-3=7
    сбои по кодам: код 404: 3
    ...

Этап 3. Большая очередь (32): задачи копятся, производители не ждут
    очередь на 32: подано 24, положено 24, взято 24, успешно 21, сбоев 3, глубина очереди 24 и
    без потерь: да, повторов страниц: нет, разных страниц 24 из 24, все потоки завершились: да
    распределение по потребителям: hw13-consumer-1=7, hw13-consumer-2=8, hw13-consumer-3=9

Этап 4. Сравнение ёмкостей
    малая очередь (ёмкость 4): глубина 4, блокировок put 12, ожидание 1669.0 мс, 25.3 стр./с
    большая очередь (ёмкость 32): глубина 24, блокировок put 0, ожидание 0.0 мс, 30.7 стр./с
    вывод: ограниченная очередь включает обратное давление (put ждёт), большая — копит задачи,

Этап 5. Итог конвейера
    все адреса обработаны: да
    страниц с 404 обработано как сбой, а не потеряно: да
    потребителей, реально бравших задачи: 3 из 3
    запросов к сайту: 48

[tpm] Демонстрация завершена: все задачи обработаны, потерь и повторов нет, производители и по

живых потоков ДЗ 13 после завершения: []
```

`48` запросов к сайту — это 24 страницы × 2 прогона (маленькая и большая очередь); каждый прогон скачивает весь список заново.

4. Промежуточные результаты по этапам

Этап	Фактический результат
Очередь на 4	24 задачи, глубина 4/4, **12 блокировок `put` , 1669 мс ожидания**, 949.1 мс, 25.3 стр./с
Очередь на 32	глубина 24/32, **0 блокировок**, 781.3 мс, 30.7 стр./с
Целостность	положено 24 = взято 24, разных страниц 24, повторов нет
Сбои страниц	3 × `404` — обработаны как результаты, конвейер не остановился
Распределение работы	все 3 потребителя участвовали (8/9/7 и 7/8/9)
Остановка	все потоки завершились в срок, живых потоков `[]`

5. Почему ограниченная очередь, «таблетки» и один результат на задачу

- **Ограниченная ёмкость.** `LinkedBlockingQueue` без ёмкости (`Integer.MAX_VALUE`) не создаёт обратного давления: производитель вычитывает весь источник в память и «убегает» вперёд. С ёмкостью `put` блокируется, и скорость конвейера определяется потребителями — именно это и видно в цифрах (12 блокировок на 24 задачи при ёмкости 4).
- **`put` / `take`, а не `offer` / `poll` с проверкой.** `offer` / `poll` не блокируются и требуют цикла ожидания — это busy-wait, который жжёт CPU. `BlockingQueue.put/take` усыпляют поток и будят его ровно тогда, когда появилось место или задача.
- **«Таблетки».** Флаг «работа окончена» не даёт гарантии, что очередь пуста: потребитель может выйти раньше, чем дообработает накопленное. Сигнал в очереди идёт после всех задач и будит каждого потребителя ровно один раз.
- **Один результат на задачу.** Результаты кладутся в `ConcurrentLinkedQueue`, а не в общий список: потребители завершаются в произвольном порядке, и обычный `ArrayList` потерял бы записи или упал. Отсчёт `положено = взято` и проверка различных номеров страниц доказывают отсутствие потерь и дубликатов.
- **Сбой страницы — результат.** `404` и обрыв соединения не выбрасывают исключение наружу: создаётся `PageResult` с `ok=false` и кодом. Иначе одна плохая страница остановила бы весь конвейер.
- **Именованные не-демон потоки.** `hw13-producer-N` / `hw13-consumer-N` видны в журнале и дампе; потоки не демоны, поэтому забытое ожидание завершения было бы заметно, а не замаскировано демон-потоками. `CrawlServer` ждёт их с таймаутом и прерывает при превышении.

6. Автоматические проверки

```
mvn -pl app test -Dtest='ru.tsu.tpm.vacancyparser.hw13.**'
```

Результат: `Tests run: 33, Failures: 0, Errors: 0, Skipped: 0`.

Тест	Что проверяет
`CrawlServerTest` (7)	нет потерь и дубликатов; сбой источника → результат-сбой, конвейер продолжается; работу берут несколько потребителей, потоки завершаются; пустой список сразу останавливает конвейер; малая очередь заполняется и блокирует `put`, большая — нет; некорректные параметры; сквозной прогон с локальным сайтом и заголовками
`TaskQueueTest` (5)	FIFO и глубина; полная очередь блокирует `put`, ожидание в счётчиках; «таблетки» по числу потребителей; некорректные параметры; описание задачи для журнала
`VacancySiteServerTest` (6)	коды и заголовки страниц; HTML разбирается тем же правилом; задержка применяется; загрузчик различает HTTP-ошибку и отсутствие сети; освобождение порта; некорректные параметры
`TitleExtractorTest` (5)	простой заголовок, регистр/атрибуты/переводы строк, сущности, отсутствие тега, нормализация

Тест	Что проверяет
`QueueReportTest` (5)	сводка и целостность; сравнение ёмкостей; примеры заголовков по номерам; сбои по кодам; производные числа
`Hw13PropertiesBindingTest` (2)	значения по умолчанию и переопределение `app.hw13.*`
`BlockingQueueDemoConditionTest` (3)	демо только при `--app.hw=hw13`

Сборка целиком: `mvn -pl app package` — успешно. Полный прогон всех домашних заданий: `Tests run: 545, Failures: 0, Errors: 0, Skipped: 0`.

7. Соответствие чек-листу

Пункт задания	Где подтверждено	Отметка
Функционал с `BlockingQueue`	`TaskQueue` на `LinkedBlockingQueue`	✅
Сервер задач	`CrawlServer`	✅
Несколько producer'ов кладут URL	3 производителя, разбиение по кругу	✅
Consumer'ы берут и обрабатывают	3 потребителя, `PageConsumer`	✅
Скачивают страницу и извлекают заголовок	`PageFetcher` + `TitleExtractor`, примеры в выводе	✅
Отчёт с этапами	Этот файл, разделы 2–4	✅
Дополнительно: задачи не теряются	`без потерь: да`, повторов нет	✅

8. Как запустить и сдать

```
cd "Теория и практика многопоточности"

# Полная демонстрация (завершается с кодом 0)
mvn -pl app spring-boot:run -Dspring-boot.run.arguments="--app.hw=hw13 --server.port=0"

# Больше страниц и потребителей
mvn -pl app spring-boot:run -Dspring-boot.run.arguments="--app.hw=hw13 --app.hw13.pages=60 --a

# Готовый jar
mvn -pl app package
java -jar app/target/vacancy-parser.jar --app.hw=hw13 --server.port=0
```

Параметры: `app.hw13.pages`, `app.hw13.producers`, `app.hw13.consumers`, `app.hw13.queue-capacity`, `app.hw13.comparison-capacity`, `app.hw13.page-delay-millis`, `app.hw13.request-timeout-millis`, `app.hw13.shutdown-timeout-millis`, `app.hw13.sample-titles`, `app.hw13.trace-lines`.

Сдача: скопировать отчёт в Google/Яндекс Документ, приложить исходники или ссылку на репозиторий, открыть доступ и прикрепить ссылку в личном кабинете.

9. Примечание о числах и окружении

Все числа — фактические замеры на машине с 12 ядрами (OpenJDK 21.0.13). Число блокировок `put` и глубина очереди зависят от планировщика и потому меняются между запусками (в разных прогонах было 12–17 блокировок при ёмкости 4), но качественная картина устойчива: маленькая очередь упирается в ёмкость и заставляет производителей ждать, большая принимает весь список сразу.